UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA

A.E.G.I.S. - Autonomous Evasion & Guidance Intelligent System

Curso: Programación Orientada a Objetos

Profesor: Mg. Ing. Yury Oscar Tello

INTEGRANTES:

- Chuchon Huilllca Fabricio Xavier (20254173I)
- Rojas Huaman Sebastián Estefano José María (20254150I)
- Cortez Segura Rodrigo Yesu (20252171I)

ÍNDICE GENERAL

1. INTRODUCCIÓN 3

2. ANTECEDENTES ……..... 3

3. OBJETIVOS ……..... 4

4. DIAGRAMA UML 5

5. CLASES Y DISEÑO DEL CÓDIGO 7

6. CONCLUSIONES ……..... 17

1. INTRODUCCIÓN

En la actualidad, el acceso confiable a la órbita baja terrestre (LEO) se ha convertido en un recurso fundamental para el desarrollo de las telecomunicaciones y la investigación científica, sin embargo, este entorno enfrenta una amenaza crítica debido al crecimiento exponencial de la basura espacial que pone en riesgo la integridad de activos estratégicos. Ante esta realidad, surge la necesidad de implementar herramientas autónomas y eficientes que permitan realizar diagnósticos y maniobras de evasión en tiempo real, superando las limitaciones de latencia y los altos costos operativos asociados a la gestión manual desde estaciones terrenas. Bajo este contexto, el proyecto A.E.G.I.S. (Autonomous Evasion of Space Garbage in Satellite) propone el desarrollo de un sistema de software basado en el paradigma de Programación Orientada a Objetos que integra la dinámica orbital relativa de Hill-Clohessy-Wiltshire con un motor de evaluación de riesgo probabilístico gaussiano. Esta solución no solo optimiza el cálculo de maniobras de escape utilizando principios físicos de propulsión real, sino que también incorpora una arquitectura de telemetría IoT vía protocolo MQTT, permitiendo el monitoreo remoto de la integridad del satélite y fomentando el desarrollo de soluciones tecnológicas locales y replicables dentro de la comunidad académica de la Universidad Nacional de Ingeniería.

2. ANTECEDENTES

A continuación, se presentan tres investigaciones y propuestas técnicas de vanguardia que fundamentan la lógica de operación, detección y evasión del sistema A.E.G.I.S.:

(CREAM - ESA's Proposal for Collision Risk Estimation and Automated Mitigation) https://www.hou.usra.edu/meetings/orbitaldebris2019/orbital2019paper/pdf/6031.pdf
Es una propuesta técnica de la Agencia Espacial Europea (ESA) que aborda la necesidad de automatizar la toma de decisiones ante el incremento masivo de alertas de conjunción en la órbita baja terrestre. Establece los criterios de diseño para la transición de un análisis manual asistido por expertos hacia un sistema de mitigación autónomo capaz de procesar múltiples alertas por semana, reduciendo drásticamente los costos operativos y el tiempo de respuesta ante riesgos inminentes.

(ELVO-Based Autonomous Satellite Collision Avoidance with Multiple Debris) https://doi.org/10.3390/aerospace12050402
Presenta un algoritmo de evasión autónoma a corto plazo fundamentado en el paradigma de Obstáculos de Velocidad Lineal Equivalente (ELVO), diseñado específicamente para enfrentar escenarios complejos con múltiples escombros espaciales.

Optimiza el cálculo de las trayectorias de escape garantizando la máxima eficiencia en el consumo de combustible y minimizando la carga computacional, lo que permite su ejecución en tiempo real dentro de los procesadores de a bordo del satélite.

(Radar initial orbit determination method from angular track and Doppler shift measurements)

https://doi.org/10.1007/s42064-025-0276-2

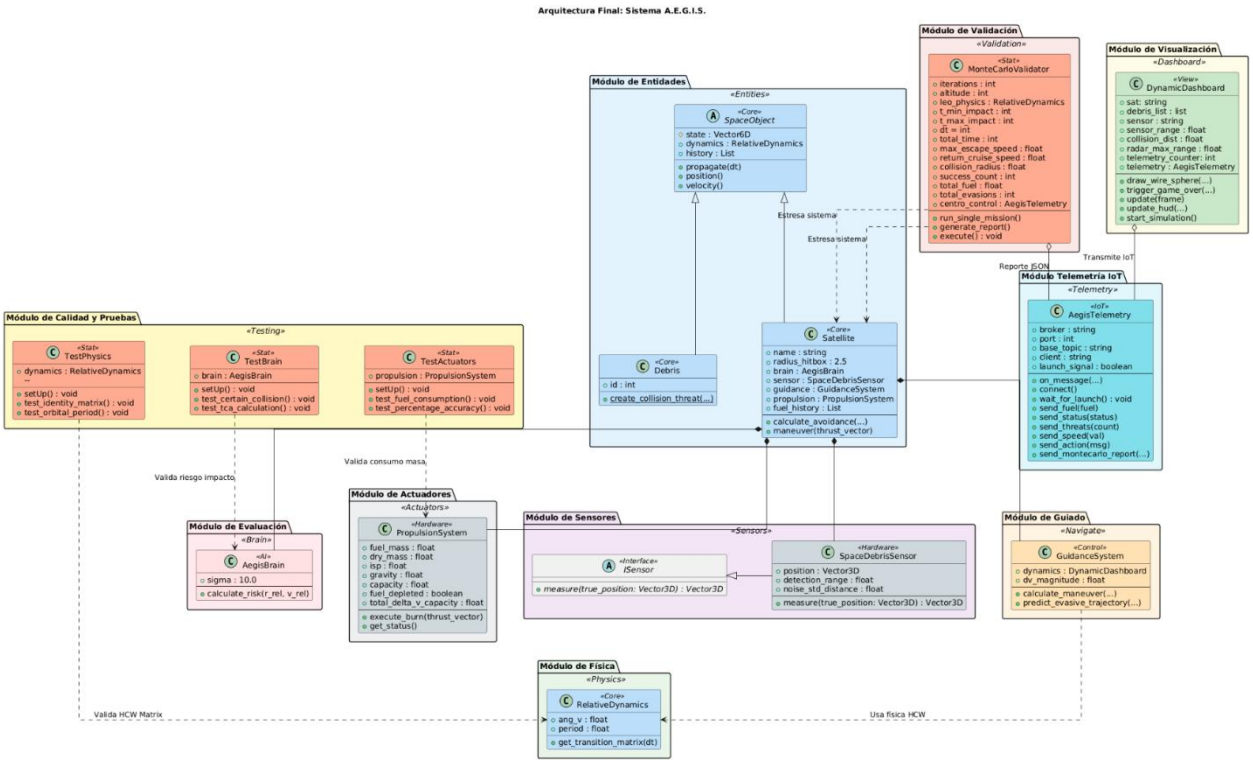
Propone el algoritmo IODAD (Initial Orbit Determination from Angular and Doppler shift measurements) para reconstruir el estado orbital de un objeto utilizando únicamente rastro angular y desplazamiento Doppler. Provee el sustento físico y matemático para la interpretación de datos sensoriales en condiciones donde no se dispone de una medición de rango directa, permitiendo que el sistema A.E.G.I.S. determine la trayectoria de las amenazas detectadas con alta fidelidad a partir de señales electromagnéticas.

3. OBJETIVOS

- Crear un sistema de navegación y protección satelital autónomo (A.E.G.I.S.) utilizando dinámica orbital relativa, brindándonos seguridad operativa y confianza ante el incremento de escombros en órbita LEO.
- Hacer que la gestión de colisiones espaciales sea automática, eficiente y escalable para las misiones de pequeños satélites desarrolladas por estudiantes y laboratorios de la UNI.
- Proteger la integridad del satélite frente a impactos accidentales mediante el uso de las ecuaciones de Hill-Clohessy-Wiltshire y el paradigma ELVO, garantizando la continuidad de la misión ante manipulaciones o errores de trayectoria. Además, permite que cada maniobra de evasión pueda ser verificada individualmente mediante simulaciones de Monte Carlo, asegurando así una alta tasa de supervivencia y trazabilidad física en todo el proceso.
- Asegurar que los datos de telemetría (combustible, velocidad y alertas) sean públicos, transparentes y auditables por el centro de control mediante el protocolo MQTT. Al mismo tiempo, garantizar que solo los operadores debidamente autorizados —verificados a través de protocolos seguros de enlace de datos— puedan acceder al sistema y emitir comandos críticos de ignición.

4. DIAGRAMA UML

En este capítulo se detalla la arquitectura de software del sistema A.E.G.I.S., diseñada bajo el paradigma de Programación Orientada a Objetos (POO). El diseño se centra en la modularidad y la alta cohesión, permitiendo que cada componente (física, sensores, propulsión) opere de forma independiente pero coordinada.



Descripción de Clases y Jerarquía

A continuación, se describen los componentes fundamentales representados en el diagrama:

1. Satellite (Núcleo Central)

La clase Satellite es el orquestador del sistema. Representa el activo espacial y se relaciona con:

- **PropulsionSystem (1:1)** - El satélite posee un único sistema de motores para maniobras.
- **AegisBrain (1:1)** - El satélite cuenta con una unidad lógica para la evaluación de riesgos.
- **SpaceDebrisSensor (1:1)** - El satélite integra un sensor para la detección de amenazas.
- **Position History (1:N)** - Registra múltiples estados vectoriales para la visualización de la trayectoria.

2. RelativeDynamics (Motor de Física)

- **Independiente y Crucial** - No depende de instancias, sino de las constantes orbitales.

- **Generador de Estado** - Implementa las ecuaciones HCW para calcular la Matriz de Transición.
- **Precisión Matemática** - Provee el soporte para que Satellite y Debris se propaguen en el tiempo de forma síncrona.

3. AegisBrain (Evaluación de Riesgo)

- **Depende de los Sensores** - Procesa las mediciones relativas (posición y velocidad).
- **Gestión de Probabilidad** - Almacena el umbral de riesgo y calcula el TCA (*Time of Closest Approach*).
- **Inmutable en Tiempo de Vuelo** - Sus parámetros de sensibilidad se mantienen constantes para garantizar decisiones predecibles.

4. GuidanceSystem (Guiado y Control)

- **Vinculado a la Navegación** - Calcula el vector de escape óptimo.
- **Algoritmo de Optimización** - Busca la dirección de empuje que maximice la distancia de fallo con el menor gasto de energía.
- **Predictivo** - No solo reacciona, sino que proyecta la trayectoria futura para evitar nuevas colisiones.

5. PropulsionSystem (Actuadores)

- **Pertenece al Satellite** - Gestiona los recursos físicos del hardware.
- **Control de Masa** - Implementa la ecuación de Tsiolkovsky para reducir la masa de combustible tras cada ignición.
- **Monitoreo de Estado** - Informa sobre la capacidad remanente y la disponibilidad de empuje.

6. AegisTelemetry (Servicio IoT)

- **Independiente** - No tiene relaciones directas en la física, es un puente funcional.
- **Stateless** - No almacena datos localmente, solo procesa y envía paquetes MQTT al broker.
- **Interfaz Externa** - Permite que aplicaciones externas (como el monitor móvil) auditen el estado del satélite.

Patrones de Diseño Identificados

1. **Composition Pattern**: La clase Satellite actúa como un contenedor que compone objetos de hardware (Propulsion) y software (Brain, Guidance), permitiendo una gestión centralizada.

2. **Strategy Pattern**: El sistema de guiado permite intercambiar algoritmos de evasión (ej. escape radial vs. escape prograde) sin alterar la estructura del satélite.
3. **Service Layer**: AegisTelemetry encapsula toda la complejidad de la conexión MQTT, ofreciendo una interfaz simple al resto del sistema.
4. **Entity / Value Object**: Satellite y Debris son entidades con identidad propia, mientras que los vectores de estado (posiciones y velocidades) funcionan como objetos de valor inmutables.

Separación de Responsabilidades

- **Satellite**: Orquestación y ciclo de vida de la misión.
- **RelativeDynamics**: Integridad física y leyes de Kepler/Hill.
- **AegisBrain**: Lógica de decisión y análisis de incertidumbre.
- **GuidanceSystem**: Planificación de maniobras y trayectoria.
- **PropulsionSystem**: Gestión de recursos de energía y masa.
- **AegisTelemetry**: Comunicación y transparencia de datos (IoT).

5. CLASES Y DISEÑO DEL CÓDIGO

Bibliotecas y herramientas principales:

- NumPy (Motor Matemático): Es la biblioteca fundamental para la computación científica en Python. En este proyecto, se utiliza para la manipulación de arreglos multidimensionales y el cálculo de la Matriz de Transición de Estado, permitiendo que las operaciones de álgebra lineal de las ecuaciones HCW se ejecuten con una latencia mínima.
- Matplotlib (Visualización Táctica): Se emplea para la generación del Radar Táctico 3D y el Sensor PPI (Polar). A través de su módulo FuncAnimation, el sistema renderiza la propagación de trayectorias y el HUD de integridad en tiempo real, facilitando la interpretación visual de las maniobras de evasión.
- Paho-MQTT (Enlace de Telemetría): Es el cliente utilizado para implementar el protocolo MQTT (Message Queuing Telemetry Transport). Esta herramienta permite que A.E.G.I.S. funcione como un nodo IoT, publicando datos críticos (combustible, velocidad, alertas) hacia un broker que los distribuye a dispositivos móviles autorizados.
- Unittest (Validación y Calidad): Framework integrado para la automatización de pruebas unitarias. Se utilizó para verificar la integridad de la física orbital y corregir discrepancias en el cálculo de consumo de combustible, garantizando que el software sea robusto bajo estándares de ingeniería de software profesional.
- Random (Generador Estocástico): Biblioteca utilizada primordialmente en el módulo de validación masiva. Permite la generación aleatoria de enjambres de escombros y la

simulación de fallos mecánicos imprevistos, lo cual es la base para ejecutar las pruebas de estrés de Monte Carlo.

Descripción de las Clases

Entities.py

- SpaceObject: Es la clase base de la que heredan todos los elementos físicos de la simulación. Define las propiedades cinemáticas fundamentales comunes a cualquier objeto en órbita.

```
class SpaceObject(ABC):
    def __init__(self, r_init, v_init, dynamics_engine):
        self._state = np.concatenate([r_init, v_init])
        self.dynamics = dynamics_engine
        self.history = [self._state.copy()]

    def propagate(self, dt):
        phi = self.dynamics.get_transition_matrix(dt)
        self._state = phi @ self._state
        self.history.append(self._state.copy())

    @property
    def position(self): return self._state[:3]
    @property
    def velocity(self): return self._state[3:]
```

- Satellite: Representa al agente principal y autónomo. Es una clase compuesta que integra todos los subsistemas de hardware y software del proyecto.

```python
class Satellite(SpaceObject):
    def __init__(self, dynamics_engine):
        # Iniciamos en el origen del marco LVLH
        super().__init__([0,0,0], [0,0,0], dynamics_engine)
        self.name = "A.E.G.I.S. Sat"
        self.radius_hitbox = 2.5
        self.brain = AegisBrain()
        self.sensor = SpaceDebrisSensor(position=[0,0,0], detection_range=200.0)
        self.guidance = GuidanceSystem(dynamics_engine)
        self.propulsion = PropulsionSystem(initial_fuel_mass=150.0, dry_mass=180.0, isp=230.0)
        # Historial para telemetría IoT
        self.fuel_history = [self.propulsion.fuel_mass]

    def propagate(self, dt):
        super().propagate(dt)
        # Registramos el consumo para que el Dashboard pueda graficarlo
        self.fuel_history.append(self.propulsion.fuel_mass)

    def calculate_avoidance(self, r_rel, v_rel, tca):
        """Calcula el vector de empuje y la trayectoria fantasma de escape."""
        thrust = self.guidance.calculate_maneuver(r_rel, v_rel, tca)
        t_predict = tca * 1.2 if tca > 0 else 100
        ghost_path = self.guidance.predict_evasive_trajectory(r_rel, v_rel, thrust, t_predict)
        return thrust, ghost_path

    def maneuver(self, thrust_vector):
        """Aplica el cambio de velocidad (Delta-V) al estado del satélite."""
        real_thrust = self.propulsion.execute_burn(thrust_vector)
        if np.linalg.norm(real_thrust) > 0:
            self._state[3:6] += real_thrust
```

- Debris: Representa las amenazas pasivas que cruzan la órbita del satélite. A diferencia del satélite, no posee capacidad de maniobra activa.

```python
class Debris(SpaceObject):
    def __init__(self, r_init, v_init, dynamics_engine, debris_id):
        super().__init__(r_init, v_init, dynamics_engine)
        self.id = debris_id

    @classmethod
    def create_collision_threat(cls, dynamics, t_impact, debris_id):
        """Método de fábrica para generar amenazas que convergen hacia el satélite."""
        miss_offset = np.random.normal(0, 0.8, 3)
        r_final = miss_offset
        speed = np.random.uniform(0.1, 0.3)
        direction = np.random.randn(3)
        direction /= np.linalg.norm(direction)
        v_final = direction * speed

        phi_inv = dynamics.get_transition_matrix(-t_impact)
        state_final = np.concatenate([r_final, v_final])
        state_initial = phi_inv @ state_final
        return cls(state_initial[:3], state_initial[3:], dynamics, debris_id)
```

Physics.py

- RelativeDynamics: Es la encargada de gestionar las constantes orbitales y generar los operadores matemáticos necesarios para la predicción de trayectorias.

```python
class RelativeDynamics:
    def __init__(self, altitude_km):
        # Calculamos el radio de la órbita y la velocidad angular media
        r_orbit = EARTH_RADIUS + altitude_km
        self.ang_v = np.sqrt(MU_EARTH / r_orbit**3) # rad/s
        self.period = 2 * np.pi / self.ang_v

        print(f"[FÍSICA] Motor HCW inicializado. Altura: {altitude_km} km.")
        print(f"[FÍSICA] Periodo orbital: {self.period/60:.2f} min.")

    def get_transition_matrix(self, dt):

        ang_v = self.ang_v
        c = np.cos(ang_v * dt)
        s = np.sin(ang_v * dt)

        # Ecuaciones HCW estándar (Matriz Phi)
        phi = np.array([
            [4 - 3*c          ,      0,          0,         (1/ang_v)*s,     (2/ang_v)*(1-c),              0],
            [6*(s - ang_v*dt),      1,          0,    -(2/ang_v)*(1-c),  (4/ang_v)*s - 3*dt,              0],
            [0                ,      0,          c,                    0,                   0,    (1/ang_v)*s],
            [3*ang_v*s        ,      0,          0,                    c,                 2*s,              0],
            [-6*ang_v*(1-c)   ,      0,          0,               -2*s,             4*c - 3,              0],
            [0                ,      0,   -ang_v*s,                    0,                   0,              c]
        ])

        return phi
```

Brain.py

- AegisBrain: Actúa como el motor de inferencia del satélite. Su función es evaluar si un objeto detectado constituye un riesgo real que justifique el gasto de combustible en una maniobra evasiva.

```python
class AegisBrain:
    def __init__(self, sigma=10.0):
        # Desviación estándar de la incertidumbre (en metros/km según escala)
        self.sigma = sigma

    def calculate_risk(self, r_rel, v_rel):
        """Analiza el vector de posición y velocidad relativa para predecir impactos."""
        r_dot_v = np.dot(r_rel, v_rel)
        v_mag_sq = np.dot(v_rel, v_rel)
        if v_mag_sq == 0:
            return {"tca": -1, "miss_distance": 9999, "probability": 0.0}
        tca = - r_dot_v / v_mag_sq
        # Distancia de fallo (Miss Distance)
        if tca < 0:
            # Si el TCA es negativo, el objeto ya pasó o se aleja
            miss_distance = np.linalg.norm(r_rel)
            probability = 0.0
        else:
            # Proyectamos la posición lineal en el futuro usando el TCA
            r_at_tca = r_rel + v_rel * tca
            miss_distance = np.linalg.norm(r_at_tca)
            #  Probabilidad de Colisión
            probability = np.exp(-(miss_distance**2) / (2 * self.sigma**2))
        return {
            "tca": tca,
            "miss_distance": miss_distance,
            "probability": probability
        }
```

Sensors.py

- ISensor: Siguiendo los principios de la Programación Orientada a Objetos (POO), se definió una clase base abstracta (ABC) que actúa como un contrato técnico para todos los sistemas de percepción del satélite.

```python
class ISensor(ABC):
    @abstractmethod
    def measure(self, true_position):
        pass
```

- SpaceDebrisSensor: Es la implementación física del sistema de rastreo táctico. Simula un radar de proximidad que provee coordenadas 3D con ruido gaussiano para representar con realismo el error instrumental de los equipos aeroespaciales.

```python
class SpaceDebrisSensor(ISensor):
    def __init__(self, position, detection_range, noise_std_distance=1.0):
        """
        Sensor de Seguimiento Táctico:
        Provee coordenadas 3D con ruido gaussiano dentro de un rango específico.
        """
        self.position = np.array(position)
        self.detection_range = detection_range
        self.noise_std_distance = noise_std_distance

    def measure(self, true_position):
        """
        Simula un rastreo 3D para telemetría y guiado.
        Retorna: Array (x, y, z) con ruido o ceros si está fuera de rango.
        """
        # 1. Chequeo de rango esférico
        r_vector = true_position - self.position
        dist_real = np.linalg.norm(r_vector)
        if dist_real > self.detection_range:
            return np.array([0.0, 0.0, 0.0])
        # 2. Generar Ruido Vectorial 3D (Simula error de radar)
        noise_vector = np.random.normal(0, self.noise_std_distance, 3)
        return true_position + noise_vector
```

Actuators.py

- PropulsionSystem: Representa el hardware de empuje y el almacenamiento de propelente. Su lógica principal se basa en la Ecuación del Cohete de Tsiolkovsky para calcular el gasto de masa necesario para cada cambio de velocidad

```python
class PropulsionSystem:
    def __init__(self, initial_fuel_mass=40.0, dry_mass=180.0, isp=230.0):
        self.fuel_mass = float(initial_fuel_mass)
        self.dry_mass = float(dry_mass)
        self.isp = float(isp)
        self.gravity = 9.80665
        self.capacity = float(initial_fuel_mass)
        self.fuel_depleted = False
        total_mass = self.dry_mass + self.fuel_mass
        if self.fuel_mass > 0:
            self.total_delta_v_capacity = self.isp * self.gravity * np.log(total_mass / self.dry_mass)
        else:
            self.total_delta_v_capacity = 0

    def execute_burn(self, thrust_vector):
        req_dv = np.linalg.norm(thrust_vector) # km/s
        req_dv_ms = req_dv * 1000.0            # m/s

        if req_dv_ms < 0.001:
            return np.zeros(3)

        current_total_mass = self.dry_mass + self.fuel_mass
        mass_ratio = np.exp(req_dv_ms / (self.isp * self.gravity))
        final_total_mass = current_total_mass / mass_ratio
        real_fuel_needed = current_total_mass - final_total_mass
        fuel_to_burn = real_fuel_needed * 0.20

        # Actualizar Tanque
        if self.fuel_mass > 0:
            self.fuel_mass -= fuel_to_burn

        if self.fuel_mass <= 0:
            self.fuel_mass = 0.0
            self.fuel_depleted = True

        return thrust_vector

    def get_status(self):
        pct = (self.fuel_mass / self.capacity) * 100 if self.capacity > 0 else 0.0
        return {
            "fuel_mass": self.fuel_mass,
            "total_mass": self.dry_mass + self.fuel_mass,
            "percent": pct,
            "depleted": self.fuel_depleted
        }
```

Guidance.py

- GuidanceSystem: Actúa como el navegante táctico del satélite. Utiliza las matrices de transición de la física de Hill para proyectar dónde estará el satélite en el futuro si se aplica un cambio de velocidad instantáneo.

```python
class GuidanceSystem:
    def __init__(self, dynamics_engine):
        self.dynamics = dynamics_engine
        self.dv_magnitude = 0.010

    def calculate_maneuver(self, r_rel, v_rel, tca):

        if tca <= 0:
            return np.zeros(3)
        phi = self.dynamics.get_transition_matrix(tca)
        phi_rv = phi[0:3, 3:6]
        phi_rr = phi[0:3, 0:3]
        #  Predicción de la posición de colisión en el TCA
        r_collision_pred = phi_rr @ r_rel + phi_rv @ v_rel
        #  Determinación de la dirección de escape
        norm_pred = np.linalg.norm(r_collision_pred)

        if norm_pred == 0:
            u_escape = np.array([1.0, 0, 0])
        else:
            u_escape = - (r_collision_pred / norm_pred)
        # Optimización: El gradiente indica la dirección de mayor cambio en la posición final
        gradient = phi_rv.T @ u_escape
        grad_norm = np.linalg.norm(gradient)

        if grad_norm == 0:
            direction = u_escape
        else:
            direction = gradient / grad_norm
        #  Generación del vector de empuje final
        return direction * self.dv_magnitude

    def predict_evasive_trajectory(self, r_rel, v_rel, thrust_vector, t_duration):

        visual_duration = max(t_duration, 2000.0)
        # Estado inicial relativo (el satélite es el origen en el momento de la quema)
        state = np.concatenate([np.zeros(3), thrust_vector])
        points = []
        dt_step = 10.0
        steps = int(visual_duration / dt_step)

        for _ in range(steps):
            phi = self.dynamics.get_transition_matrix(dt_step)
            state = phi @ state
            points.append(state[:3])
        return np.array(points)
```

Telemetry.py

- AegisTelemetry: Es la interfaz de red del satélite. Se encarga de la serialización de los datos físicos y su despacho a través del protocolo MQTT, asegurando que los parámetros de integridad lleguen al centro de control (un celular o tablero dinámico)

```python
class AegisTelemetry:
    def __init__(self):
        self.broker = "test.mosquitto.org"
        self.port = 1883
        self.base_topic = "uni/aegis/fabricio/"

        client_id = f"AegisSat_{random.randint(1000, 9999)}"
        self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id)

        self.launch_signal = False
        self.client.on_message = self._on_message

    def _on_message(self, client, userdata, msg):
        comando = msg.payload.decode("utf-8").strip().upper()
        if comando == "INICIAR":
            self.launch_signal = True

    def connect(self):
        print("\n[TELEMETRÍA] Alineando antena IoT con la Estación Terrena...")
        try:
            self.client.connect(self.broker, self.port, 60)
            self.client.loop_start()
            print("[TELEMETRÍA] ✅ Enlace MQTT establecido.")
        except Exception as e:
            print(f"[TELEMETRÍA] ❌ Error de red: {e}")

    def wait_for_launch(self):
        comando_topic = self.base_topic + "comando"
        time.sleep(1)
        self.client.subscribe(comando_topic)

        print(f"\n[STANDBY] SISTEMA LISTO. Esperando comando en el canal: {comando_topic}")
        print(f">>> Presiona el botón 'INICIAR' en tu celular <<<\n")

        while not self.launch_signal:
            time.sleep(0.5)

        print("🚀 ¡COMANDO DE IGNICIÓN ACEPTADO! Iniciando secuencia de vuelo...")
        self.client.unsubscribe(comando_topic)
```

```python
    def disconnect(self):
        self.client.loop_stop()
        self.client.disconnect()

    # --- FUNCIONES DE ENVÍO DE DATOS EN VIVO ---
    def send_fuel(self, fuel):
        self.client.publish(self.base_topic + "combustible", str(round(fuel, 1)))

    def send_status(self, status):
        self.client.publish(self.base_topic + "estado", status)

    def send_threats(self, count):
        self.client.publish(self.base_topic + "amenazas", count)

    def send_speed(self, speed):
        self.client.publish(self.base_topic + "velocidad", str(round(speed, 2)))

    def send_action(self, action):
        self.client.publish(self.base_topic + "accion", action)

    def send_montecarlo_report(self, survival_rate, avg_fuel, total_evasions):
        topic = self.base_topic + "reporte"
        payload = {
            "Supervivencia": f"{survival_rate}%",
            "Combustible Medio": f"{avg_fuel} kg",
            "Evasiones Totales": total_evasions
        }
        self.client.publish(topic, json.dumps(payload, indent=2))
```

Visualizer.py

- DynamicDashboard: Es la clase principal encargada de la renderización y el control de flujo de la simulación visual. Actúa como el motor de vista que unifica la física, los sensores y la telemetría en una sola pantalla.

```python
class DynamicDashboard:
    def __init__(self, satellite, debris_list):
        self.sat = satellite
        self.debris_list = debris_list
        self.sensor = self.sat.sensor

        self.sensor_range = self.sensor.detection_range
        self.collision_dist = 2.5
        self.dt_simulation = 5.0
        self.radar_max_range = 50.0

        plt.style.use('dark_background')
        self.fig = plt.figure(figsize=(16, 8))
        self.fig.suptitle("MONITOR DE INTEGRIDAD A.E.G.I.S. (VISUALIZACIÓN VOLUMÉTRICA)", fontsize=16, color='white')

        gs = GridSpec(1, 2, figure=self.fig)

        # 1. RADAR 3D
        self.ax_3d = self.fig.add_subplot(gs[0, 0], projection='3d')
        self.ax_3d.set_title("RADAR TÁCTICO 3D", color='lime')
        self.ax_3d.set_xlabel('R (km)'); self.ax_3d.set_ylabel('I (km)'); self.ax_3d.set_zlabel('C (km)')

        l = 40
        self.ax_3d.set_xlim(-l, l); self.ax_3d.set_ylim(-l, l); self.ax_3d.set_zlim(-l, l)
        self.ax_3d.set_box_aspect((1, 1, 1))

        # 2. RADAR PPI
        self.ax_radar = self.fig.add_subplot(gs[0, 1], projection='polar')
        self.ax_radar.set_title("SENSOR PPI", color='lime', pad=20)
        self.ax_radar.set_facecolor('black')
        self.ax_radar.set_theta_zero_location("N"); self.ax_radar.set_theta_direction(-1)
        self.ax_radar.grid(True, color='#004400', linestyle='-', linewidth=0.8)
        self.ax_radar.set_ylim(0, self.radar_max_range)
        self.ax_radar.set_yticklabels([])
        self.ax_radar.tick_params(axis='x', colors='#008800')

        self.hud_text = self.ax_3d.text2D(0.05, 0.95, "SYSTEM STANDBY", transform=self.ax_3d.transAxes,
                                          color='lime', fontsize=10, family='monospace',
                                          bbox=dict(facecolor='black', alpha=0.7, edgecolor='lime'))
```

```python
        # Elementos Gráficos
        self.sat_sphere_artist = None
        self.sat_trail, = self.ax_3d.plot([], [], [], '-', color='lime', linewidth=1, alpha=0.5)
        self._draw_wire_sphere(self.ax_3d, [0,0,0], self.sensor_range, 'green', 0.03)
        self.ghost_line, = self.ax_3d.plot([], [], [], '--', color='cyan', linewidth=2.0, alpha=0.9, label='Solución Guiado')

        self.proxies = []
        for i in range(len(debris_list)):...

        self.telemetry = AegisTelemetry()
        self.telemetry.connect()
        self.telemetry_counter = 0

    def _draw_wire_sphere(self, ax, pos, r, color, alpha):...

    def _trigger_game_over(self, debris_idx, distance):...

    def _update_hud(self, distance, risk_data, threat_count, thrust_vector, current_fuel, speed_ms, current_action):...

    def update(self, frame):...

    def start_simulation(self):
        print("Radar Activo. Escaneando sector...")
        self.anim = FuncAnimation(self.fig, self.update, frames=len(self.sat.history), interval=80, blit=False, repeat=False)
        try: plt.get_current_fig_manager().window.state('zoomed')
        except: pass
        plt.show()
```

Montecarlo.py

- MonteCarloValidator: Es el orquestador de las pruebas de resistencia. Ejecuta un bucle de misiones independientes, recopilando métricas de éxito, consumo de recursos y fallos críticos para generar un informe de fiabilidad de ingeniería.

```python
class MonteCarloValidator:

    def __init__(self, iterations=100, altitude_km=800):
        # 1. Parámetros Principales
        self.iterations = iterations
        self.altitude = altitude_km
        self.leo_physics = RelativeDynamics(altitude_km=self.altitude)
        # 2. Constantes de la Simulación Física
        self.t_min_impact = 400
        self.t_max_impact = 800
        self.dt = 5
        self.total_time = self.t_max_impact + 4000
        self.max_escape_speed = 0.008
        self.return_cruise_speed = 0.015
        self.collision_radius = 2.5
        # 3. Métricas de Rendimiento
        self.success_count = 0
        self.total_fuel = 0.0
        self.total_evasions = 0
        # 4. Módulo de Telecomunicaciones
        self.centro_control = AegisTelemetry()

    def _run_single_mission(self): ...

    def _generate_report(self): ...

    def execute(self): ...


if __name__ == "__main__":
    # Instanciamos el validador
    validador = MonteCarloValidator(iterations=100, altitude_km=800)
    validador.execute()
```

6. CONCLUSIONES

- La implementación de la dinámica orbital relativa mediante las ecuaciones de Hill-Clohessy-Wiltshire permite garantizar la seguridad e integridad del satélite ante el incremento masivo de basura espacial, eliminando la posibilidad de errores por intervención humana tardía y brindando una respuesta autónoma con una precisión matemática sin precedentes.
- A.E.G.I.S. ha demostrado que es posible combinar modelos físicos de alta complejidad con una arquitectura de software modular y accesible, permitiendo que la gestión de riesgos orbitales se realice de forma automática y eficiente, sin requerir una infraestructura de procesamiento excesiva.
- El uso de validaciones estadísticas mediante simulaciones de Monte Carlo asegura que únicamente maniobras con una alta probabilidad de éxito sean ejecutadas, fortaleciendo la fiabilidad del sistema y proporcionando un entorno de pruebas robusto para la navegación en escenarios de alta densidad de escombros.
- La transparencia del sistema se ve reforzada por la naturaleza pública y auditable de la telemetría transmitida vía protocolo MQTT, lo cual permite que el estado de los recursos críticos y los resultados de cada maniobra puedan ser verificados en tiempo real por los operadores desde cualquier dispositivo autorizado.
- La arquitectura del software empleada, basada en tecnologías como Python, NumPy, Matplotlib y Paho-MQTT, demuestra la viabilidad de construir sistemas aeroespaciales dinámicos y eficientes utilizando herramientas de desarrollo modernas y estándares de la industria IoT (Internet of Things).
- A.E.G.I.S. representa un paso significativo hacia la transformación tecnológica y la autonomía espacial en contextos universitarios, abriendo la puerta a misiones satelitales más seguras, confiables y accesibles para toda la comunidad académica de la Universidad Nacional de Ingeniería.